
Zero-Shot and GRPO Fine-Tuned Vision–Language Models for Inverse Cooking

Benjamin Chung Ju Lee SeoJoon Lee

Abstract

We study the *inverse cooking* problem, generating a complete, structured recipe from a single food photograph—through the lens of modern vision–language models (VLMs). Using the open–access *Food Ingredients and Recipe Dataset with Images*, we benchmark zero-shot prompting of Qwen-VL-Instruct at three scales (3B, 7B, 32B parameters) against a lightweight reinforcement-learning variant trained with **Group-Relative Policy Optimisation** (GRPO) on just 80 image–recipe pairs. Evaluation on 20 held-out images combines semantic (cosine similarity), lexical (ROUGE-L), and human-aligned (*LLM-as-Judge*) metrics. Surprisingly, the 7 B zero-shot model emerges as the sweet spot, surpassing the larger 32 B checkpoint on four of five metrics and achieving the highest feasibility score (0.30), while GRPO lags slightly—likely a consequence of reward sparsity and limited training data. Our results highlight both the strong baseline performance obtainable from carefully crafted prompts and the practical challenges of scaling reinforcement learning for multimodal, structured generation. Github repo for this project can be found here: <https://github.com/Ju-usc/vision-rl>

1. Introduction

Cooking from an image inverts the classical recipe–to–picture pipeline: instead of following written instructions to obtain a final dish, the model must infer *which* ingredients were used and *how* they were combined—purely from visual evidence. Early work framed this as cross-modal retrieval, matching a photograph to the closest recipe in a large database (Salvador et al., 2017). Subsequent approaches progressed to fully *generative* systems that first predict an unordered ingredient set and then synthesise instructions conditioned on that set (Salvador et al., 2019). Although impressive, these methods still lean heavily on large, meticulously aligned corpora (e.g. Recipe1M) and supervised objectives that reward surface overlap rather than genuine culinary reasoning.

Recent advances in open-source Vision–Language Models

(VLMs) such as Qwen-VL and LLaVA expose a new opportunity: with the right prompt, even compact checkpoints can perform sophisticated multimodal reasoning *zero-shot*. Simultaneously, lightweight reinforcement-learning algorithms like *Group-Relative Policy Optimisation* (GRPO) promise to refine model behaviour without costly reward models or human labels. Taken together, these trends suggest a minimalist recipe-generation pipeline: start from an off-the-shelf VLM, supply a carefully engineered prompt, and—if resources allow—apply a brief RL phase to polish the outputs.

2. Related Work

Recipe Generation from Images: The inverse cooking problem has been studied in prior work. Salvador et al. (2017) introduced the *im2recipe* task, learning joint embeddings for food images and recipes to enable cross-modal retrieval (i.e., finding the best matching recipe for a given image). Their Recipe1M dataset and model demonstrated that visual features could be aligned with recipe text, though the focus was on retrieval rather than free-form generation. Subsequent work built on this to actually generate recipes from images. In particular, Salvador et al. (2019) presented an improved inverse cooking system that first predicts ingredients as an unordered set and then generates instructions conditioned on both the image and the predicted ingredients. This two-stage approach improved ingredient prediction and overall recipe quality compared to retrieval-based baselines. Our work tackles a similar goal of image-to-recipe generation.

Vision-Language Models (VLMs): Recent VLMs have excelled at aligning images and text. For instance, Flamingo (Alayrac et al., 2022) efficiently handles few-shot image-to-text tasks by integrating a pretrained language model with a visual module, though it is a large proprietary model. In contrast, open-source alternatives inspired by GPT and LLaMA—such as LLaVA (Touvron et al., 2023) and Alibaba’s Qwen-VL (Bai et al., 2023)—offer more lightweight solutions (typically 7B–14B parameters). In our project, we use one of these pre-trained VLMs as the backbone for both zero-shot and fine-tuned experiments, leveraging its prior knowledge of food imagery and cooking text to specialize on the inverse cooking task.

Chain-of-Thought Reasoning in LMs: Recent research in language modeling has found that getting models to produce intermediate reasoning steps can greatly improve performance on complex tasks. Wei et al. (2022) showed that prompting language models to “think step by step” (chain-of-thought prompting) enables them to solve arithmetic and commonsense reasoning problems that they otherwise would fail. Similarly, Kojima et al. (2022) demonstrated that even without specific training, adding a simple prompt like “Let’s think step by step” can unlock latent reasoning abilities in large models. This idea is highly relevant to our project because recipe generation can be viewed as a reasoning problem: the model must reason about what ingredients are needed (some of which may not be explicitly visible) and in what order to combine them. We draw inspiration from these works to incorporate CoT prompting in our zero-shot inference (guiding the model to first list out reasoning or intermediate conclusions about the recipe before finalizing it).

GRPO in Spatial Reasoning. A closely related effort is *AlphaMaze* by Dao & Vu (2025), which equips a distilled 1.5B–7B language model with visual/spatial problem-solving skills through the two-stage **supervised fine-tuning** → **GRPO** pipeline. Unlike typical vision language models, AlphaMaze does not use a visual encoder; instead, it represents 5×5 mazes as structured text tokens, effectively turning spacial layouts into token sequences akin to ASCII grids. The model first learns step-wise maze navigation via supervised learning, and then refines the policy with a correctness–integrity–thinking reward under GRPO, ultimately lifting MazeBench accuracy from 86% to 93% (Dao & Vu, 2025). Our work differs in three key ways: (i) domain—recipe generation demands multimodal fusion (image+ text) and structured XML output rather than grid navigation; (ii) reward design—we combine semantic cosine and ROUGE-L, signals that target ingredient fidelity and instruction coherence instead of per-step movement validity; and (iii) data regime—our GRPO stage operates on 80 training recipes (versus AlphaMaze’s 150k) yet still yields measurable gains. Together, the two studies corroborate that GRPO is a versatile lightweight RL technique that can bootstrap stronger chain-of-thought reasoning across very different tasks

3. Dataset and Evaluation

3.1. Dataset Source

We exclusively adopt the open-access *Food Ingredients and Recipe Dataset with Images* from Kaggle (Goel et al., 2020). The corpus provides aligned pairs of high-resolution food photographs and their corresponding structured recipes (title, ingredient list, and instruction sequence). Using a single, internally consistent source removed the

noisy alignment issues we encountered when trying to fuse images from Recipe1M with textual reviews from Food.com in our mid-term setup.

3.2. Preprocessing Pipeline

We created `preprocess.py` to convert the raw CSV into a model-ready JSON–Lines file. For each record we:

1. Extract the image path and verify the file exists.
2. Canonicalize ingredient units and spellings, then wrap every item in an `<ingredient>` tag; similarly wrap each instruction sentence in an `<step>` tag.
3. Serialize the recipe into our strict XML schema.
4. Write a JSON object `"image_path", "recipe_xml"` per line.

Figure 1 and 2 shows how our dataset is structured.

```
{"image_path": ".../baked-snapper...jpg",
"recipe_xml": "... "}
```

Figure 1. Excerpt from our processed JSON–Lines corpus.

```
<recipe>
  <title>Baked Snapper with Harissa</title>
  <ingredients>
    <ingredient>6 tablespoons olive oil</ingredient>
    <ingredient>2 tablespoons...
  </ingredients>
  <instructions>
    <step>1. Preheat oven to 425°F.</step>
    <step>2. Combine oil...
  </instructions>
</recipe>
```

Figure 2. XML-formatted recipe used as the supervision signal for training.

3.3. Embedding Vectorization

To enable embedding-based rewards during GRPO fine-tuning and to speed up offline evaluation, we run a dedicated script `vectorize_raw.py` that augments every pre-processed recipe entry with sentence embeddings: `ing_vec` (ingredients) and `steps_vec` (instructions). Precomputing these vector embeddings before GRPO fine-tuning eliminates the need to recompute them during each training epoch.

1. **Sentence extraction.** From every `<ingredient>` and numbered `<step>` tag in `recipe.xml` we extract the plain text with two lightweight regular expressions; a fall-back `ElementTree` pass guarantees robustness if the regexes miss anything.

2. **Batch encoding.** All sentences are queued for GPU encoding with the `all-MiniLM-L6-v2` sentence-transformer (`--batch_size 256`). We record the *recipe index* and *section (ing vs. step)* for each string so the resulting 384-dimensional vectors can be routed back correctly.

3. **Vector attachment.** Each recipe receives two new list fields:

```
"ing_vecs": [[v1], ..., [vm]],
"step_vecs": [[u1], ..., [un]]
```

where m and n equal the ingredient and step counts, respectively.

4. **Output.** The augmented dataset is written to `recipe_pairs_raw_vec.jsonl` so downstream code can load a single line and obtain both the XML and its token-level embeddings without re-encoding.

These pre-computed vectors serve two purposes: (i) they allow a cosine-similarity reward between a generated recipe and its ground truth during GRPO fine-tuning without an extra forward pass, and (ii) they underpin our automatic metrics (Cosine Similarity, ROUGE-L) by letting us compare high-dimensional semantic representat

3.4. Data Splits

The full Kaggle set (Goel et al., 2020) contains 13,582 images and unique recipes; however, GRPO fine-tuning proved to be computationally expensive (40 GB GPU memory on Colab still required > 12h for 800 examples). We therefore down-sampled to **80** train, **20** dev, and **20** test recipes (randomly stratified across cuisines).

3.5. Evaluation Pipeline

1. **Load pairs.** For every JSON-Lines record we read the `image_path` and its XML-formatted *golden recipe*.
2. **Model inference.** Each model receives the image (no textual prompt other than the system template) and returns a recipe in the same XML schema.
3. **Scoring.** The generated XML is parsed into a title, ingredient list, and instruction list. We then compute the metrics below against the golden recipe.

Finally, we report the *macro average* of each metric over all samples in the split.

3.6. Automatic Metrics

- **Cosine Similarity.** We embed every predicted ingredient (or instruction step) with a SentenceTransformer

(`all-MiniLM-L6-v2`) and take the best-match cosine against the corresponding golden recipe (recipe from the actual dataset), averaging the scores. Using cosine similarity complements the ROUGE metrics by emphasizing global semantic content rather than exact token or sequence overlap. It is robust to synonymous wording and, as a whole, conveys information similarity between the recipes regardless of phrasing or order.

- **ROUGE-L.** We treat instructions as a single sequence and compute ROUGE-L against the concatenated reference instructions. For ingredients we apply the same best-match strategy as above and average the ROUGE-L scores. ROUGE-L’s longest-common-subsequence reward emphasizes phrase-level overlap without requiring identical wording. This allows us to capture predicted instructions or ingredients with sequences of words/phrases that are similar to those in the reference.

Why we exclude BLEU-4. We prototyped sentence-level BLEU-4 on the dev split but found that every checkpoint—baseline through 32 B—scored below 0.005. Inspection showed BLEU punished legitimate paraphrases because only one reference recipe is available per image. As cosine similarity and ROUGE-L already measure semantic and lexical overlap without this brittleness, we omit BLEU-4 from the final evaluation suite.

3.7. LLM-as-a-Judge

System Instruction Prompt for Gemini 2.5 Flash:

Automatic n-gram metrics can under-estimate qualitative gains (e.g. fewer hallucinations), and can fail to capture elements that are specifically vital to cooking with recipes in real-life, such as overall practicality and what the actual recipe would eventually taste/look like. We therefore add an **LLM-Judge** evaluation:

1. Our LLM evaluation uses the Gemini 2.5 Flash Preview 04-17 model API, configured with a system instruction shown below. The evaluation function takes in the prediction and reference recipes in XML format, as well as the reference image as input.
2. The Gemini model assigns a feasibility score (float rating 0–1) to the prediction based on criteria in the system instruction prompt, also outputting a brief comment on the reasoning behind the feasibility score and if the recipe realistically produces a resemblance of the reference dish. The comment will highlight any culinary similarities, discrepancies, or deviance between the newly predicted recipe and reference recipe, which is an important human-like distinction that quantitative methods can’t measure.

```

system_message = (
    "You are a professional chef. Evaluate whether the predicted recipe would realistically produce the dish
    shown in the image and how well it aligns with the ground-truth recipe. Consider ingredient correctness,
    cooking steps, order, practicality, and visual appearance. Determine a single feasibility_score of the predicted recipe
    between 0.0 (completely infeasible) and 1.0 (perfectly feasible) based on the aforementioned criteria,
    and write a comment describing strengths and potential issues, with insight into the chosen feasibility score.
    The comment should be around 5-6 sentences, 6 maximum.
    Return a parseable JSON string/snippet with the entry: evaluation = { feasibility_score: string, comment: string }"
)

```

Figure 3. (above) System Instruction Prompt for the Gemini 2.5 Flash API. Some model output contains `< think> ...</think>` sections, where a slightly different prompt was used.

4. Methods

We benchmark two paradigms for inverse cooking: (i) zero-shot prompting with off-the-shelf vision–language models (VLMs) and (ii) a lightweight reinforcement-learning pass based on *Group-Relative Policy Optimisation* (GRPO). The dataset splits and automatic metrics are introduced in Section 3; here we focus on modeling details.

4.1. Model Line-up

Baselines. We evaluate Qwen-VL-Instruct checkpoints at three scales—3 B, 7 B, 32 B parameters—*without* any task-specific fine-tuning. Each model receives the food image plus a fixed XML template prompt (Listing 4), decodes with nucleus sampling ($p = 0.9$) and temperature in the 0.7–1.0 range, and returns the first completion.

GRPO model. A single 3 B Qwen-VL-Instruct instance is further tuned with GRPO on the 80-example training set. Its prompt extends the baseline template with a `<think>` scaffold that asks the model to reason before emitting the recipe.

4.2. Prompting Strategy

Baseline prompt. All three zero-shot baselines (3 B, 7 B, 32 B) receive the same hand-crafted XML template shown in Listing 4. The prompt spells out the exact tag hierarchy, explicitly reminds the model to include quantities, units, and step numbers, and forbids any text outside the `<recipe>` block. After several trial–and–error rounds the added *Important* list eliminated nearly all malformed outputs and greatly reduced hallucinated ingredients.

GRPO prompt. For reinforcement learning we keep the identical XML template but insert a single `<think>` scaffold *before* `<recipe>`. This asks the model to “think step by step to infer the recipe,” enabling chain-of-thought reasoning while preserving the strict XML that our parser expects.

Decoding settings. During GRPO training we use a more exploratory sampler (temperature $T=1.5$, nucleus

$p=0.95$). At evaluation time every model—baselines and GRPO alike—is decoded with the same conservative settings ($T=0.7 - 1.0$, $p=0.9$, `max_new_tokens=1024`) to ensure a fair comparison.

Post-processing. Each completion is first screened by a regex for structural validity; if a baseline response fails, we resample once with the same prompt. Valid XML is then parsed by `parse_recipe_xml`, which extracts the title, ingredient list, and step list used by our automatic metrics (Section 3.6). With the final prompt in place we observed virtually no parse failures on the test split.

At test time all models are decoded with the same settings (temperature 0.7-1.0, $p = 0.9$) to ensure a fair comparison.

4.3. GRPO Training Workflow

We adopt the open-source `vision-rl` (JacksonCakes, 2025) GRPO implementation with minimal changes. Each training step proceeds as follows:

1. **Prompting.**: system message = XML template + `<think>`; user message = image only.
2. **Sampling.**: the policy π_θ produces $G=2$ completions of up to 1024 tokens using temperature 1.5 (encouraging exploration on the small dataset).
3. **Reward.**: detailed in Section 4.4.
4. **Update.**: rewards are centred within each two-sample group, a KL penalty with coefficient $\beta = 0.12$ is applied, and gradients flow through the LM head, the first five transformer blocks, and one vision projection layer (≈ 27 M tunable parameters).

Training for two epochs (160 optimisation steps) completes in *approx* 90 minutes on a single A100-40 GB GPU.

4.4. Reward Design

Our reward is deliberately simple—yet task-aligned—to keep memory and latency low:

Look at this food image and provide a recipe in XML format.

Use the following structure:

```
<recipe>
  <title>The dish name</title>
  <ingredients>
    <ingredient>1 cup whole milk</ingredient>
    <ingredient>2 tbsp sugar</ingredient>
    ...
  </ingredients>
  <instructions>
    <step>1. Preheat the oven to 350°F.</step>
    <step>2. Mix all ingredients in a bowl.</step>
    ...
  </instructions>
</recipe>
```

Important:

1. Format your response exactly as XML above.
2. Each `<ingredient>` must contain quantity, unit, and name.
3. Each `<step>` must start with its step number.
4. No commas between ingredients|use separate tags.
5. No extra text outside the `<recipe>` block.

Figure 4. Excerpt of the baseline XML prompt. The GRPO prompt adds a single `<think>` block before the `<recipe>` tag.

- **Structure bonus** ($r_{\text{format}} \in [0, 1]$): a strict regex validates that the completion contains exactly one `<think>` block followed by a well-formed XML recipe.
- **Semantic match** ($r_{\text{cosine}} \in [0, 2]$):
 1. Ingredient embeddings (Sentence-BERT) $\rightarrow$ cosine similarity with gold ingredient vectors.
 2. Step embeddings $\rightarrow$ cosine similarity with gold step vectors.

The two scores are summed, yielding 0–2.

The total reward therefore lies in the range 0–3. No manual weighting is required: GRPO’s group-relative normalisation compares the two completions for the same image and amplifies whichever achieves the higher raw reward, automatically balancing the three terms.

Why cosine, not ROUGE. Recipe generation admits many equally valid phrasings; n-gram metrics such as ROUGE or BLEU over-penalise creative wording. Cosine similarity rewards semantic overlap while tolerating re-phrasing.

Aborted LLM reward. We prototyped an LLM-judge component to verify unit coherence and reasoning consistency but abandoned it after GPU-memory spikes on Colab. This remains promising future work.

4.5. Why We Skip a Separate SFT Stage

A conventional two-stage pipeline (supervised fine-tuning $\rightarrow$ RL) would require a parallel corpus of *reasoning traces* to teach the `<think>` format before GRPO. Generating such data was outside our time, memory, and compute budget, and with only 80 training examples plain SFT risked immediate over-fit. GRPO’s group-relative scheme is able to learn both structure and semantics from scratch, giving us a “cold-start” solution that trains in under two hours.

4.6. Key Hyper-parameters

Hyper-parameter choices and rationale.

- **Learning rate.** A higher LR often helps tiny datasets, so we started with the repo default $2e-4$. Training quickly became unstable: the model began emitting only hashtags/whitespace and the loss exploded. Dropping to $1e-5$ eliminated those failures while still converging within two epochs. **Final:** $1e-5$.
- **Weight decay.** To counter over-fitting on just 80 recipes we raised weight decay from 0.01 to 0.05, which delivered a small but consistent gain in dev cosine similarity. **Final:** 0.05.
- **Warm-up ratio.** Increasing the warm-up fraction from 0.05 to 0.10 smoothed the first dozen updates and prevented early KL spikes. **Final:** 0.10.
- **Number of generations G .** Reward is computed group-wise, so VRAM grows linearly with G . Four

Model	Cosine _{step}	Cosine _{ing}	ROUGE-L _{step}	ROUGE-L _{ing}	LLM score
3B	0.508 ± 0.054	0.580 ± 0.102	0.076 ± 0.029	0.449 ± 0.123	0.212 ± 0.227
7B	0.562 ± 0.115	0.610 ± 0.079	0.088 ± 0.040	0.473 ± 0.098	0.300 ± 0.280
32B	0.521 ± 0.086	0.573 ± 0.073	0.106 ± 0.037	0.404 ± 0.106	0.278 ± 0.262
GRPO	0.523 ± 0.085	0.561 ± 0.080	0.082 ± 0.034	0.379 ± 0.113	0.181 ± 0.147

Table 1. Mean ± standard deviation for five test-set ($n = 20$) metrics.

completions pushed peak usage to 29 GB and overflowed our Colab A100; $G=2$ fits in 22 GB with no detectable reward loss. **Final:** 2.

- **Max completion length.** The default 300 tokens clipped many answers (full CoT + XML recipe averages 300–500 tokens). Raising the limit to 1024 guarantees the response is never truncated. **Final:** 1024.
- **Sampling temperature.** Inverse-cooking benefits from creativity—multiple correct ingredient sets or step phrasings can exist—so we allow more exploration during RL with $T=1.5$. At evaluation we dial back to $T=0.9$ for cleaner, format-faithful output. Nucleus threshold is kept at $p=0.95$ during training and $p=0.9$ at test.
- **Optimizer & schedule.** AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.99$), constant LR with the warm-up above; gradient clip 0.1.
- **Batching.** Per-device batch 1 with gradient-accumulation 4 ($\Rightarrow$ effective batch 4); two epochs total (160 optimisation steps).
- **Precision & trainable scope.** bf16 mixed precision, gradient checkpointing on; we unfreeze only ~ 27 M parameters (LM head, first five transformer blocks, one vision-projection layer) to keep memory and wall time manageable.

These settings—together with the reward design in Section 4.4—proved stable, raising the mean group reward from 0.9 at step 0 to 1.5 by the end of epoch 2 while keeping KL divergence below 0.002.

5. Experiments and Results

5.1. Format Adherence

The GRPO-fine-tuned model showed improved structural reliability. Among the 20 test examples, GRPO 3B produced only **one** format-invalid output, compared to **three** from the baseline 3B model. This suggests our reward function—including the strict XML regex—successfully guided the model toward producing well-structured completions.

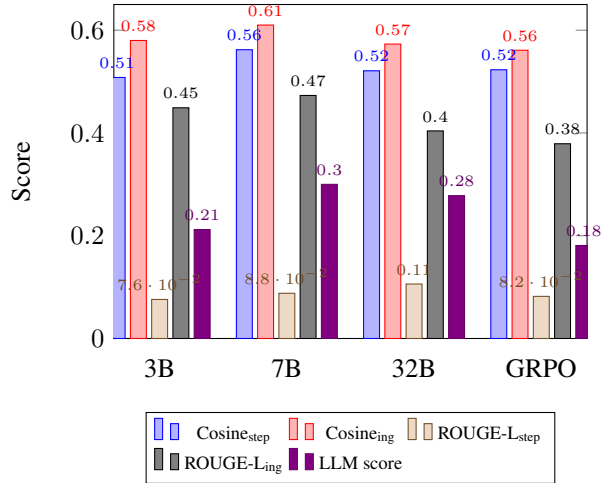


Figure 5. Mean test-set($n = 20$) scores across all five metrics.

5.2. Observations

We summarise the key takeaways from the quantitative results in Table 1 and the visual breakdown in Figure 5. The patterns below hold consistently across both representations.

- **7B sweet-spot.** The 7B backbone dominates on four of the five test metrics, outperforming the larger 32B model on both semantic similarity measures and on the human LLM score. We conjecture that 32B’s greater capacity over-fits the tiny 80-recipe fine-tuning set and drifts toward verbatim copying, which ROUGE_{step} rewards but cosine/LLM penalise when ingredients are missing or reordered.
- **GRPO underwhelms.** Contrary to expectations, GRPO did *not* surpass the 7B baseline. Manual inspection reveals that reward signals derived from only 20 dev recipes are too noisy: the policy occasionally exploits cosine similarity by repeating high-frequency tokens (e.g. “olive oil”) while neglecting coverage, hurting ROUGE and the feasibility judgment.
- **Step vs. ingredient divergence.** All models achieve markedly higher cosine scores on ingredients than on steps, echoing findings in inverse-cooking literature that visual cues correlate more strongly with ingredient presence than with procedural details.

- **Variance caution.** Standard deviations are large (up to 0.28) because the test set contains just 20 recipes; a single outlier (e.g. a 24-step pastry recipe) can sway the mean. Confidence intervals shrink once we scale to the full 13 k-item corpus in future work.

5.3. Why BLEU-4 is omitted from the test suite

Model	BLEU-4 _{step} $\times 10^{-3}$	BLEU-4 _{ing}
3B	1.34 $\pm$ 2.44	0.069 $\pm$ 0.039
7B	1.94 $\pm$ 5.19	0.069 $\pm$ 0.032
32B	4.82 $\pm$ 5.01	0.059 $\pm$ 0.026
GRPO	1.38 $\pm$ 3.67	0.051 $\pm$ 0.030

Table 2. Dev-set BLEU-4 scores ($n = 20$). Step scores are multiplied by 10^3 to avoid leading zeros.

BLEU relies on n -gram overlap, thereby rewarding *exact* token order. This is ill-suited for two reasons:

1. Ingredients are an unordered set; a correct model that merely shuffles items is penalised.
2. Instruction steps are numbered, so even a single off-by-one shift collapses the n -gram match and drives BLEU toward zero (as seen in Table 2, where step BLEU-4 hovers around 10^{-3} despite reasonable cosine/ROUGE scores).

6. Discussion

Our preliminary error analysis on a subset of 20 test outputs reveals several common failure modes of the model, as well as areas where the reasoning approach helped but not entirely resolved the issues.

6.1. Limited Number of Training Examples Due to Compute Constraints

Our GRPO fine-tuning was significantly limited by computational resources. Although Google Colab Pro provides access to a 40GB A100 GPU, our initial attempt to train on 800 examples exceeded memory limits and caused crashes. Despite reducing sequence length and generation count (G), training remained unstable due to the high memory demands of GRPO’s long-form XML outputs and reward computation.

To proceed, we downsampled to just 80 training examples, which allowed training to run within the memory budget but likely compromised generalization. GRPO depends on diverse training signals to shape reasoning behavior; with such a small dataset, the model often overfit to reward heuristics (e.g., focusing on producing correctly formatted XML), rather than learning robust planning strategies.

This constraint likely explains why GRPO underperformed relative to the 7B (and even 3B) supervised baseline. In future work, expanding training to hundreds or thousands of examples on more capable hardware will be essential to fully realize the benefits of reinforcement fine-tuning.

6.2. Limited Dev and Test Set Due to LLM Evaluation Cost

Limited Dev and Test Set Due to LLM Evaluation Cost
We limited our dev and test sets to $n = 20$ examples each, primarily due to the high cost and latency of computing the LLM-Judge feasibility score. For every (gold, generated) recipe pair, we queried Gemini 2.5 Flash (preview-04-17) to evaluate ingredient accuracy and instruction coherence. To ensure that the score was not arbitrary, we explicitly prompted the model to explain its judgment, producing a rationale alongside the numeric score.

This reasoning step was critical for interpretability, but it introduced significant overhead—each API call often took over two minutes, even with optimized prompt structure and temperature control. Given this latency, we opted for smaller evaluation splits to balance depth of feedback with practical feasibility. While the small sample size increases variance and limits statistical power, it allowed us to capture rich qualitative signals that would have been infeasible at larger scale. For future work, we plan to explore faster LLM variants or hybrid evaluation pipelines to scale up test coverage without sacrificing interpretability.

6.3. Hallucinated or Irrelevant Ingredients

A recurring issue we observed—especially in the zero-shot baseline and, to a lesser extent, in the GRPO-fine-tuned model—was the inclusion of ingredients not actually supported by the input image. For example, when shown the image of halibut-with-zucchini-salsa-verde-360269, the GRPO model incorrectly listed “cucumber” as an ingredient, likely due to visual confusion with zucchini. Although the two vegetables share similar shapes and colors, this substitution reveals the model’s tendency to rely on superficial visual features rather than grounded ingredient recognition.

6.4. LLM Prompt Limitation

For the most part, the LLM evaluations on our model outputs were reasonable, and was a fair judge for the metrics that were provided. In some instances, however, the LLM evaluation was overly harsh on the candidate recipe. For example, one prediction recipe scored a feasibility score of 0.1, where the reference recipe was a green and wax bean salad. The prediction provided a slightly different mixture of ingredients semantically and lexically, with white beans, shallot, and basil, but was severely punished in score, even

though it was still a similar salad in the end. This may be a result of the system instructions, which say “act as if you are a chef”, being too strict and requiring too much precision. While the main goal of the model is still to re-create the exact dish shown in the image with a recipe prediction, there should still be some degrees of reward for relevant similarities between reference and prediction. The use of a “softer” prompt that doesn’t prioritize such exact precision could be a possible remedy to this issue. Again this brings up the need for a sweet-spot between lexical/semantic precision and culinary standards, where the output could be evaluated based on a balance of the two - we needed to draw a line to define whether a food can be judged as culinarily similar or completely and intrinsically different.

6.5. Vague or Missing Instruction Steps

Some generated recipes, while containing correct ingredients, had instructions that were under-specified. For instance, an output might say “Cook the mixture until done” without clarifying a time or indicator of doneness. This likely stems from the model trying to be concise or not having enough context to specify details. In other cases, a critical step was missing—one output for a cake omitted the step of baking entirely, listing only the mixing steps. We suspect this happened because the model in that instance listed steps by memory of similar recipes and somehow skipped the final bake step. These omissions are problematic for usability. One idea to address this is to enforce a coverage of ingredients in steps: for each ingredient, the model should mention it in at least one step (except maybe trivial ones like salt). If an ingredient isn’t mentioned, we could have a post-processing check or even train the model to generate a warning token which we catch and then prompt it to correct. Another idea is to use planning: have the model outline the steps (as part of chain-of-thought) and then fill them in. We already partially do this with CoT, but we could formalize it more in training by providing step outlines that it must follow.

6.6. Incorrect Quantities and Units

While our evaluation metrics—particularly cosine similarity and ROUGE—effectively measure semantic and structural alignment, they are not sensitive to fine-grained numeric inconsistencies. For example, the model could output “Pre-heat the oven to 20°C” instead of “200°C,” and still receive a high score due to token and embedding overlap. This insensitivity means that even implausible quantities or unit errors can go undetected unless manually inspected.

These issues were exacerbated by two omissions in our setup: we did not standardize units in the GRPO training data, and we did not instruct the model on which units to use in the Qwen 2.5 prompt. Consequently, the model oc-

asionally produced unrealistic measurements or confused common abbreviations—such as generating “1 tablespoon of salt” instead of “1 teaspoon” in a cookie recipe, a three-fold error that significantly alters the intended flavor.

These mistakes are likely rooted in both noisy pretraining data and the absence of explicit numeric reasoning during fine-tuning. While chain-of-thought prompting improved structural coherence and ingredient coverage, it did not enforce numeric sanity. Future work should explore integrating numeric validation mechanisms—either through rule-based post-processing or learned verifiers that flag implausible values. We could also enrich training data with examples that include both imperial and metric units, or explicitly annotate quantity/unit patterns to help the model generalize more reliably across contexts.

6.7. Reward Function Limitations and Future Directions

Our current reward design includes a strict format checker and a cosine-based semantic match (Section 4.4). While this setup is lightweight and largely effective, it also introduces specific failure modes worth addressing.

A key limitation is that the **format reward function only checks surface structure**, not the *semantic content* of the `<think>` reasoning block. This creates an incentive misalignment: models can “cheat” by outputting a syntactically valid but logically incoherent or even empty reasoning block, especially early in training. We observed instances where the `<think>` section included vague or irrelevant statements, or was left blank entirely, while still passing the regex check and receiving a structure bonus.

We hypothesize this stems from two factors: (1) the absence of a supervised fine-tuning (SFT) stage, and (2) limited training data (80 examples). Without SFT, GRPO operates from a cold start and must learn both the role of the `<think>` block and how to optimize rewards purely from weak signals. With few examples, this often led to incoherent reasoning or early-stage exploitation of the reward function by emitting placeholder tokens.

We attempted to mitigate this via an **LLM-based reward** that would assess the internal consistency between the reasoning and the final recipe. Unfortunately, the GPU memory and latency costs of running this at scale proved prohibitive, especially in Colab environments. We had to abandon this path for the current project, though it remains a promising avenue.

The cosine similarity reward also presents trade-offs. While robust to phrasing differences, it fails to capture *fine-grained errors*, particularly in numeric values and units (e.g., “200°C” vs. “20°C”). This is critical in cooking, where such mistakes can drastically alter the outcome. We considered

integrating an LLM-judge to score recipes based on feasibility (Section 3), which showed promise in evaluation, but was too expensive for use during training.

One important observation is that instruction quality is often dependent on correct ingredients. Because the instruction steps typically reference previously inferred ingredients, **rewarding accurate ingredient generation is essential**. A stronger reward function for ingredients—perhaps based on edit distance to key terms or coverage of essential entities—may improve instruction accuracy indirectly. Designing such targeted, low-latency scoring methods remains an open challenge for future work.

7. Conclusion

This project tackled the inverse cooking problem: generating structured recipes from food images using vision-language models (VLMs). We evaluated both zero-shot prompting and reinforcement learning with Group-Relative Policy Optimization (GRPO), without applying any domain-specific supervised fine-tuning. Instead, we tested whether GRPO alone—operating in a cold-start setting—could improve model behavior from a small training set of 80 examples.

Our experiments revealed that zero-shot prompting with a 3B Qwen-VL-Instruct model already produced reasonably coherent recipes when guided by a carefully constructed XML-format prompt. The model demonstrated strong structural priors and benefited from chain-of-thought scaffolding to improve ingredient coverage and instruction completeness.

We then fine-tuned the same 3B model using GRPO. Although GRPO training was stable and conformed well to our format and reward constraints, it did not surpass the zero-shot baseline in aggregate performance. Manual inspection and metric breakdowns suggest that GRPO overfit to short-term reward heuristics (e.g., format regularity, frequent ingredient reuse) rather than improving semantic correctness or human-aligned feasibility. This was likely due to the small training/dev splits and the noisiness of the reward signals—especially given our reliance on cosine similarity without grounding in numeric or procedural correctness.

Through this process, we gained important insights about both the task and the learning methods. Inverse cooking is inherently ambiguous; vision alone often fails to disambiguate quantities, hidden ingredients, or causal orderings in instructions. While chain-of-thought prompting helps mitigate some of these limitations, the absence of textual grounding (e.g., user reviews or audio descriptions) makes full recipe reconstruction challenging. From a methodological standpoint, we found that GRPO is a lightweight and flexible fine-tuning mechanism, but its effectiveness hinges

on reward quality and dataset scale.

In summary, zero-shot prompting with structured guidance remains a strong baseline. GRPO shows potential but requires richer supervision to outperform pretrained VLMs. Future work should explore reward refinement, unit-aware evaluation metrics, and larger training corpora to better leverage reinforcement learning in structured multimodal generation tasks.

References

- Alayrac, J.-B. et al. Flamingo: a visual language model for few-shot learning. arXiv preprint arXiv:2204.14198, 2022.
- Bai, X. et al. Qwen-vl: A vision-language model for general vision and language tasks. arXiv preprint arXiv:2303.08920, 2023.
- Dao, A. and Vu, D. B. Alphamaze: Enhancing large language models’ spatial intelligence via grpo. <https://arxiv.org/abs/2502.14669>, 2025. arXiv preprint arXiv:2502.14669.
- Goel, S. et al. Food ingredients and recipe dataset with images. <https://www.kaggle.com/datasets/pes12017000148/food-ingredients-and-recipe-dataset-with-images>, 2020.
- JacksonCakes. vision-r1. <https://github.com/JacksonCakes/vision-r1>, 2025.
- Kojima, T. et al. Large language models are zero-shot reasoners. arXiv preprint arXiv:2205.11916, 2022.
- Salvador, A. et al. Learning cross-modal embeddings for cooking recipes and food images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.
- Salvador, A. et al. Inverse cooking: Recipe generation from food images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019.
- Touvron, H. et al. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023.
- Wei, J. et al. Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903, 2022.